Programmatic API use and structured outputs

Lesson 2 · LLM Fundamentals, Prompting & API Integration

By the end of today, you can

- **Read** a response: why it stopped, what it cost; **stream** a long one
- **Tell** a transient error from a permanent one, and **retry** to match
- **Get** a record in a fixed shape, three ways, and **compare** them
- **Enforce** rules a schema cannot express, and **re-prompt** at most twice
- **Trace** a ReAct loop, and say what stops it

Where lesson 1 left you

- Free text, and a parser that **guessed** where the label was
- Now: **111,322 reviews** nobody reads; the product team wants **records**
- Language · sentiment · aspects with quotes · defect · would recommend
- **Twenty reviews**: plain, Spanish, French, sarcastic, blank, long, five stars with a broken part

A script that crashes on the first review

```
SYSTEM = "...the five fields...  Answer with the JSON object only, ..."
r = client.messages.create(model="claude-haiku-4-5", max_tokens=1000, system=SYSTEM, messages=[...])
record = json.loads(r.content[0].text)
```

```
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

The record was right. It was in a code fence.

JSON asked for in the prompt, Claude Haiku 4.5	
Answers wrapped in ```` ```json ````	**60 / 60**
Loaded by `json.loads` as returned	**0 / 60**
Fit the schema once the fence is stripped	54 / 60
Invented an aspect (`defect_mentioned`)	6 / 60, the same two reviews every run

Measured 2 October 2026. **First the call must survive, then the answer must be trusted.**

1 · The call, from a script

What else the response tells you

Field	Why a script needs it
`stop_reason`	`max_tokens` = **cut off**; `tool_use` = asks your code; `refusal`
`usage`	the bill, per call
`_request_id`	what support asks for; log it

- **Pin** versions: an upgrade can change a default (1.11.0 already exists)
- **A pushed key:** deleting the commit does not help. **Revoke it:** tell the coordinator

2 · The same call, elsewhere

One call, two shapes

	Messages · anthropic	Chat Completions · openai
System prompt	`system="..."`	`a message, role "system"`
Output limit	`max_tokens (required)`	`max_completion_tokens (optional)`
The text	`content: a list of blocks`	`choices[0].message.content`
Why it stopped	`stop_reason: end_turn, max_tokens, tool_use, refusal`	`finish_reason: stop, length, tool_calls, content_filter`
Tokens billed	`usage.input_tokens / output_tokens`	`usage.prompt_tokens / completion_tokens`
Fixed-shape output	`output_config.format, strict tools`	`response_format`

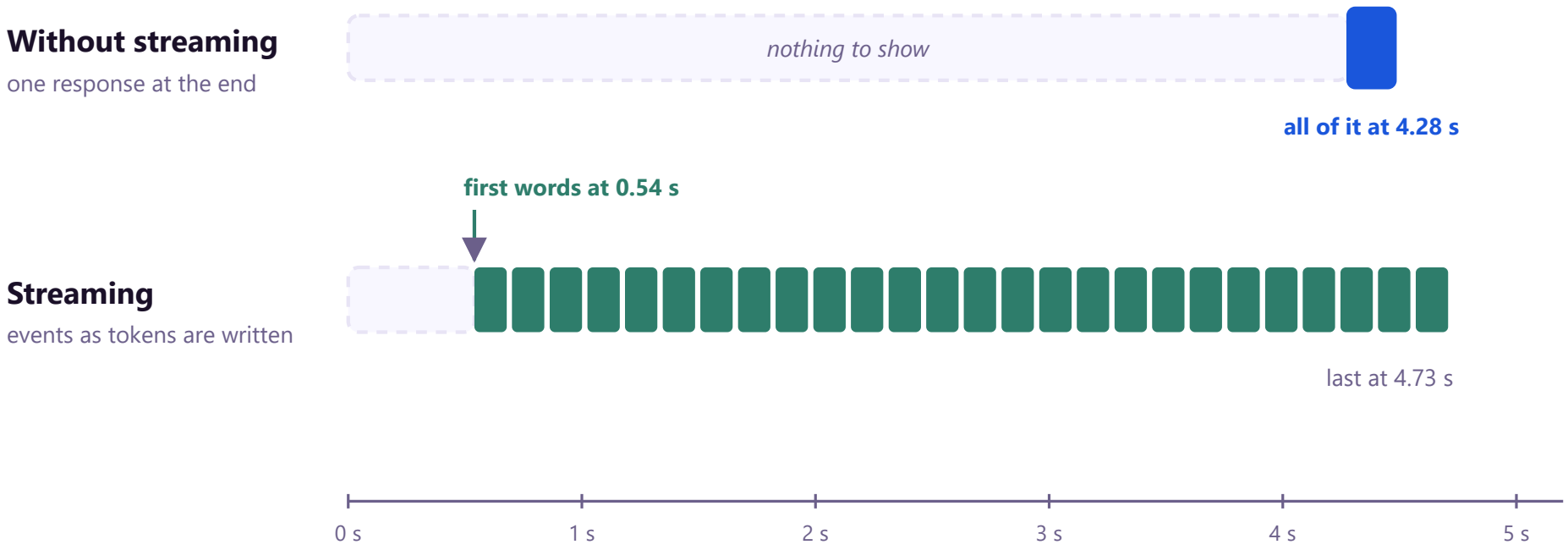
Read from the SDKs themselves on 2 October 2026: anthropic 1.10.0 and openai 3.23.0.
Mistral's API and most servers for open models follow the Chat Completions shape.

What transfers, and what does not

- **Transfers:** a request (model, conversation, limits) → a response (content, why it stopped, tokens)
- **Does not:** where the system prompt goes, where the text is, the field names
- Chat Completions is the de facto standard: **Mistral, local open models**
- Code written for one shape **fails on the other**, sometimes silently

3 · Streaming

Streaming moves the first words, not the last



Measured 2 October 2026: Claude Haiku 4.5, a 250-word summary of review 98330, 342 and 344 output tokens.
Same tokens, same price: streaming moves the first words earlier, not the last.

When to stream

- **A person is waiting:** a chat window, a summary on screen
- **A very long answer:** Anthropic recommends it past ten minutes
- **Not** a batch writing a file: nobody is watching
- Volume without hurry: **Message Batches**, half the price
- An error can arrive **after** the 200, mid-stream

4 · Errors: transient or permanent

Retry the transient, fix the permanent

Transient: wait and retry

the same request may work later

429	**RateLimitError**	`rate_limit_error`
500	**InternalServerError**	`api_error`
529	**OverloadedError**	`overloaded_error`
—	**APITimeoutError**	the request timed out
—	**APIConnectionError**	the connection dropped

Permanent: fix the request

it will fail the same way every time

400	**BadRequestError**	`invalid_request_error` (1)
401	**AuthenticationError**	`authentication_error`
403	**PermissionDeniedError**	`permission_error`
404	**NotFoundError**	`not_found_error`
413	**RequestTooLargeError**	`request_too_large`

(1) A spend limit on your key also returns a 400, not a 429: "bad request" can mean the budget is spent.

Classes from the anthropic SDK 1.10.0; status codes and types from Anthropic's errors page, 2 October 2026.

Two traps

- **A spend limit returns a 400**, not a 429: "bad request" can mean the budget is gone
  (through OpenRouter: a 403 for the key's limit, a 402 for the account's credit)
- **A spend-cap 429 has no** `retry-after`: it keeps failing until someone raises the cap
- Catch by **class**, most specific first; never match the message
- A 400 sent again is **the same 400**

5 · Retries and backoff

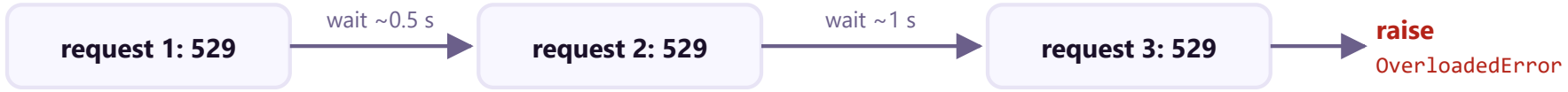
The SDK already retries

- Connection errors, timeouts, **408, 409, 429, 5xx**
- **Twice** by default, waiting 0.5 s up to 8 s, honouring `retry-after`
- `anthropic.Anthropic(max_retries=5)`, or `max_retries=0` to switch it off

Fake API answering 529	Requests
The SDK as installed	3
`@retry(stop_after_attempt(3))` on top	**9**

Retries: one layer, never two

One layer: the SDK's default, two retries



3 requests, 1.38 s, then the error reaches your code

Two layers: @retry(stop_after_attempt(3)) on that client



9 requests for one call. On a 400, tenacity alone re-sends the same bad request 3 times.

Measured 2 October 2026 against a fake server that always answers 529 (anthropic 1.10.0, tenacity 9.1.4).
Waits shown are the SDK's starting values; they grow, with jitter, up to 8 s.

One layer: yours, when you need one

```
client = anthropic.Anthropic(max_retries=0)

@retry(retry=retry_if_exception(transient),
       wait=wait_random_exponential(multiplier=0.5, max=8),
       stop=stop_after_attempt(4), reraise=True)
def call(**kwargs):
    return client.messages.create(**kwargs)
```

- **Backoff** grows the wait · **jitter** spreads clients · **the cap** stops it
- Another SDK, or plain HTTP, may have **no retries at all**: the policy is yours

6 · Structured output

The shape, as a Pydantic model

```python
class AspectMention(BaseModel):
    aspect: Aspect                                  # one of twelve
    polarity: Literal["positive", "negative", "mixed"]
    evidence: str                                   # copied exactly

class ReviewRecord(BaseModel):
    language: Literal["en", "es", "fr", "de", "other"]
    sentiment: Literal["positive", "negative", "mixed", "neutral"]
    aspects: list[AspectMention]
    defect_mentioned: bool
    would_recommend: Optional[bool]                 # only if stated
```

The **ids stay in your code**: a model asked to copy one can mistype it.

Three ways to get a record

JSON asked for in the prompt

`"Answer with the JSON object only"`

```
```json
{ "language": "en", ...
```
```

json.loads: 0 / 60 load

input 298 tokens
output 177 tokens
+42 input over the plain call

$1.18
per 1,000 reviews

A strict tool

`tools=[...], strict: true`

```
tool_use block
input: { "language": ...
}
```

valid: 20 / 20

input 1,287 tokens
output 180 tokens
+1,031 input over the plain call

$2.19
per 1,000 reviews

Structured outputs

`output_config.format / parse()`

```
text block
{ "language": "en", ...
}
```

valid: 60 / 60

input 984 tokens
output 115 tokens
+728 input over the plain call

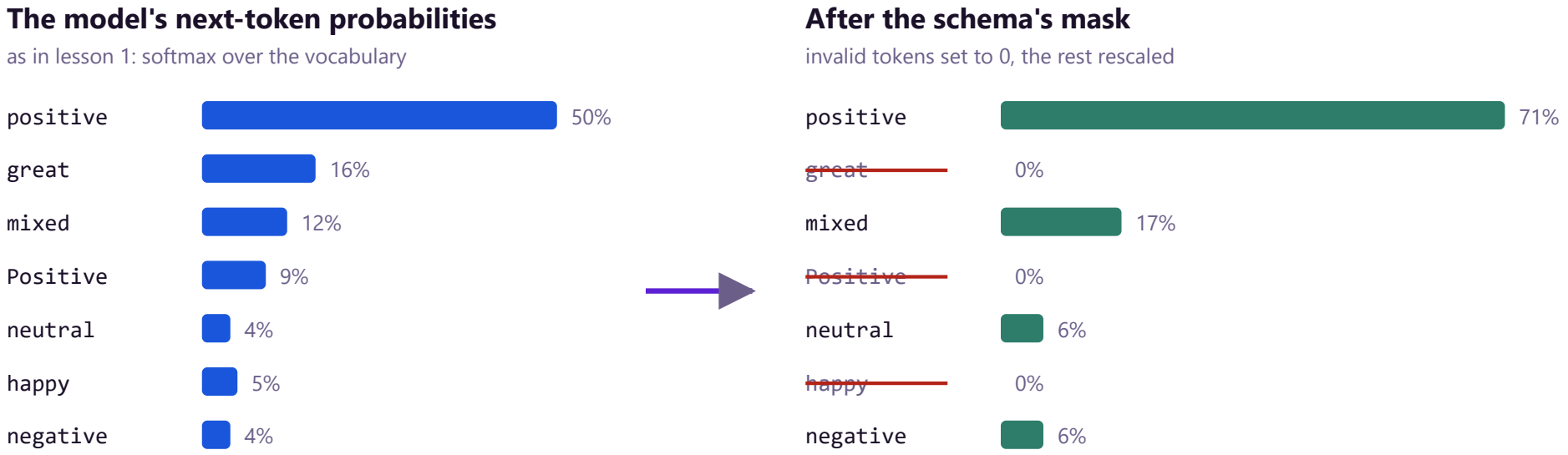
$1.56
per 1,000 reviews

Measured 2 October 2026: Claude Haiku 4.5, the same 20 reviews (three runs except the tool, one); medians per review. The plain request is 256 input tokens. Prices $1 / $5 per million input / output tokens. The tool route adds Anthropic's tool system prompt (588 tokens when the tool is forced).

Constrained decoding: the schema removes tokens before sampling

Written so far:
`{"language": "en", "sentiment": "`

The schema allows here:
`positive | negative | mixed | neutral`



Illustrative probabilities: the shape of the mechanism, not one model's numbers.
Sampling then proceeds as in lesson 1, only among the tokens the schema still allows.

What it does not guarantee

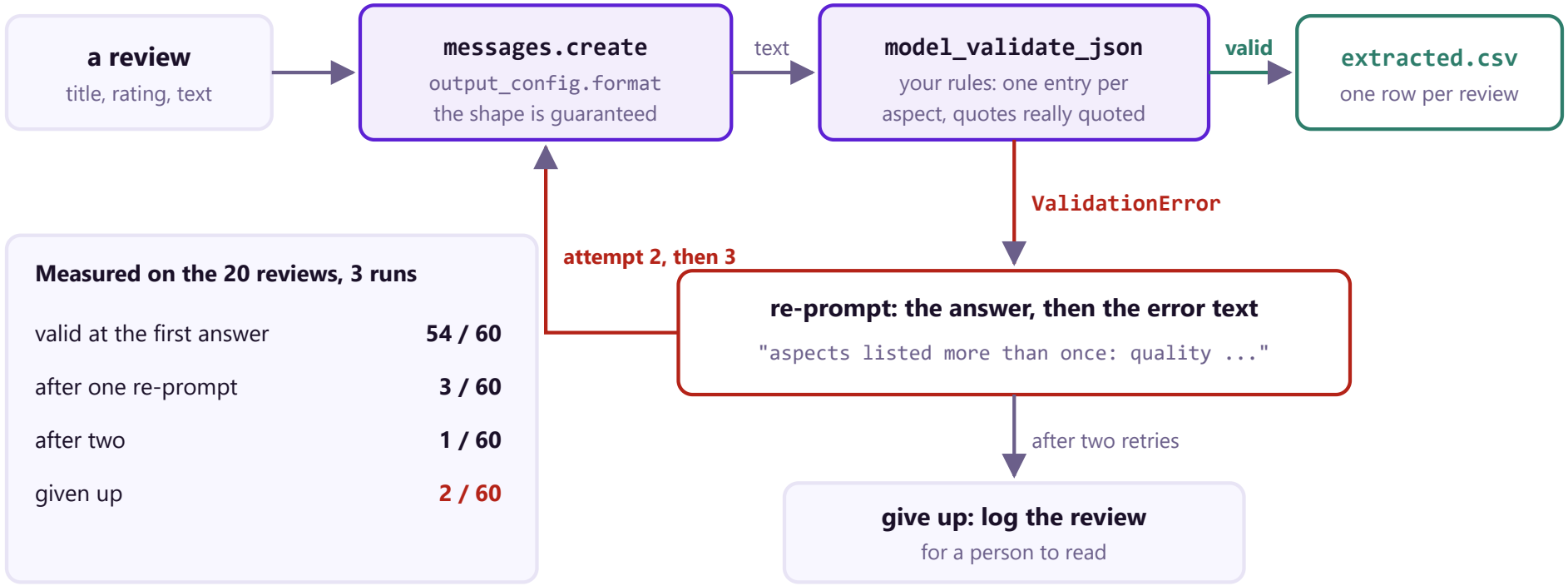
- A **refusal** or a **cut-off** answer: check `stop_reason` first
- `minLength`, `minimum`…: dropped from what the model sees
- A hand-built strict tool needs `"additionalProperties": false` (a 400 otherwise)
- **The shape, not the content.** A valid record is one that loads

7 · Validation, and re-prompting

Valid, and wrong

- Through structured outputs: **15 of 60 records listed an aspect twice**
- Review 143272: `quality +`, `quality −`, `quality +` → counted three times
- No JSON schema can say "each aspect once"
- **Pydantic can:** a `model_validator`; and "every quote is in the review": a `field_validator` with **context**

Validate in your code, re-prompt with the error, give up after two



Measured on the 20 reviews, 3 runs

valid at the first answer	**54 / 60**
after one re-prompt	**3 / 60**
after two	**1 / 60**
given up	**2 / 60**

Measured 2 October 2026: Claude Haiku 4.5, the settled schema, rules in the prompt and in Pydantic validators.
Re-prompting costs a whole call each time: the cap is what keeps a rule the model cannot meet from billing forever.

Why not `parse()` for this

- `parse()` validates **inside the SDK**
- A rule fails → raised there, **after** the tokens are billed
- The text and `usage` are lost: nothing to re-prompt with
- So: `messages.create` + `anthropic.transform_schema(Model)` + `model_validate_json(text, context=...)`

What no validator catches

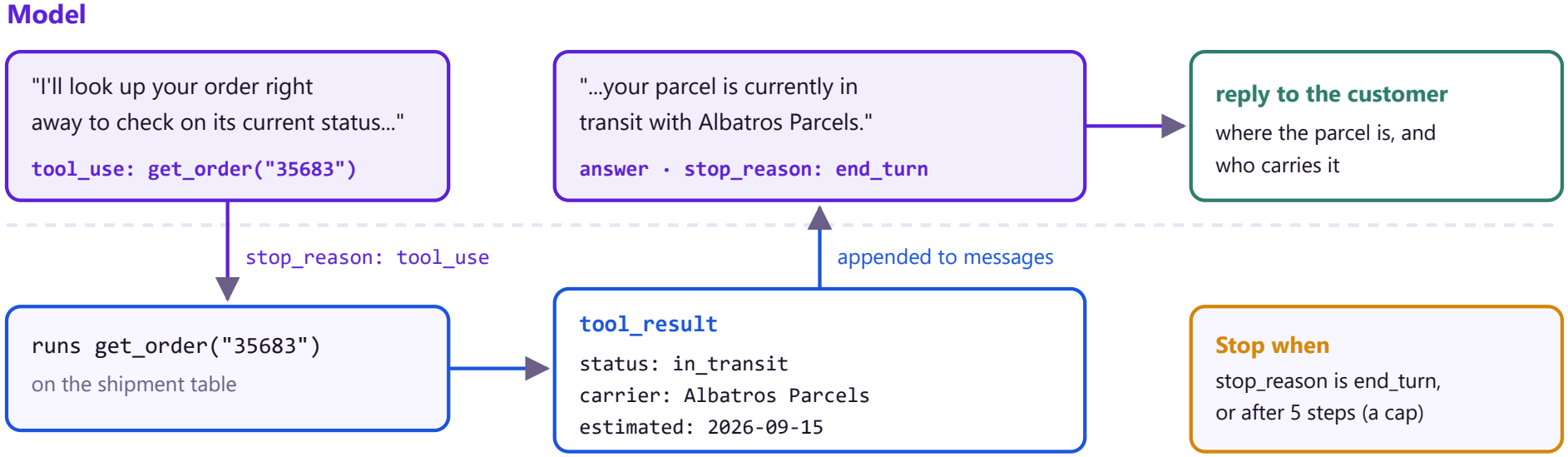
- **13 of 20 reviews** changed record between runs: every version valid
- 154546, "burned my eyes": a defect in one run, not in two
- Blank text, title "Five Stars": a positive record, **no aspects** (right)
- `if text:` lets a single space through; `text.strip()` does not
- **Only a checked record says which is right:** Exercise 1, then lesson 4

8 · ReAct

Reason, act, observe, repeat

- Yao et al. (2022): reasoning traces and actions **"in an interleaved manner"**
- Lesson 1's chain-of-thought + today's tools
- Ticket T022: *"Can you tell me where the parcel is right now?"*
- The answer is in the shipment table, not in the ticket

ReAct: think, ask for a tool, read the result, repeat



One loop: call the model, run the tool it asked for, append the result, call again.

The model never runs anything: it writes a request; your code decides whether to run it.

Ticket T022 of tickets-v1.0.0: "Where is my order 35683?". Shipment data from the company's release v1.0.0.
Measured 2 October 2026: Claude Haiku 4.5, 2 calls, 1,772 input and 433 output tokens. ReAct: Yao et al., 2022.

The model asks. Your code acts.

- `stop_reason: tool_use` → **your code** runs `get_order`, sends a `tool_result`
- Each step is a **full call** that resends the conversation: **cap it**
- No loop needed when you know what to fetch: paste it in
- Every agent framework runs **this loop underneath**

Now

1. **The demo:** every number of today, live, and a model that looks up an order
2. **The quiz:** twenty questions, twenty minutes
3. **Exercise 1:** write `extract.py`, checked step by step, in pairs
4. **At home:** Exercise 2, audit a client an assistant wrote; and the project's second step